UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

Escuela Profesional de Ciencias de la Computación



LENGUAJES Y COMPILADORES

FLOWXYRUST

LABORATORIO N.º 2

Definición de la gramática del lenguaje

Curso:	Lenguajes y Compiladores
Profesor:	Ruiz De la Cruz Melo, Carlos Augusto
Integrantes:	Sotelo Atuncar, Diego Alejandro Rodriguez Claeyssen, Isaac Aaron Emilio Denis, Cabello Villon
Fecha:	Septiembre de 2026

Lima – Perú

2026

1. Elementos usados en las estructuras

Tomamos los siguientes elementos del Laboratorio 1 como referencia para construir las producciones. Esta tabla conserva sus nombres originales; más adelante presentamos los símbolos que usamos en la gramática.

Elemento	Representa
<ID>	Un identificador válido. Ejemplo: edad.
<TIPO_BASICO>	Un tipo definido directamente por FlowxyRust. Ejemplo: entero32.
<TIPO_ENTERO>	Un tipo entero con o sin signo. Ejemplo: entero32 o natural32.
<TIPO>	Un <TIPO_BASICO> o una clase definida por el programador. Ejemplo: entero32 o Persona.
<VALOR>	Un valor escrito directamente. Ejemplo: 14, 20.5, 'D', "Diego" o verdadero.
<VALOR_VECTOR>	Un valor completo de vector. Su forma será [<VALORES>]. Ejemplo: [10, 20, 30].
<VALOR_OBJETO>	Un valor completo de objeto. Su forma será <CLASE> { <VALORES_ATRIBUTOS> }. Ejemplo: Persona { nombre: "Diego", edad: 20 }.
<EXPRESION>	Una combinación válida que produce un valor. Ejemplo: a + b.
<OPERANDO>	Un elemento que puede participar en una expresión. Ejemplo: 10, edad o notas[0].
<DESTINO>	Un elemento de tipo básico al que se puede asignar un valor. Ejemplo: edad, notas[0] o persona.edad.
<INDICE>	Una expresión entera usada para acceder a una posición de un vector. Ejemplo: 0 o i.
<CONDICION>	Una expresión cuyo resultado es de tipo booleano. Ejemplo: edad >= 18.
<CLASE>	El identificador de una clase declarada. Ejemplo: Persona.
<OBJETO>	El identificador de un objeto declarado. Ejemplo: persona.
<ATRIBUTO>	El identificador de un atributo definido en una clase. Ejemplo: edad.
<METODO>	El identificador de un método definido para una clase. Ejemplo: obtenerEdad.
<ATRIBUTOS>	Uno o más atributos declarados dentro de una clase.
<METODOS>	Uno o más métodos declarados dentro de un bloque implementa.
<VALORES>	Uno o más valores separados mediante comas. Ejemplo: 10, 20, 30.
<SENTENCIAS>	Cero o más instrucciones válidas dentro de un bloque.

Usamos el terminal id para representar los nombres de variables, clases, objetos, atributos, métodos y funciones.

2. Palabras reservadas

Mantenemos las palabras reservadas del Laboratorio 1. No podrán utilizarse como identificadores.

Categoría	Palabras reservadas
Declaración	`sea, constante`
Modificador	`mutable`
Orientación a objetos	`clase, implementa, este`
Tipos enteros	`entero16, entero32, entero64, natural16, natural32, natural64`
Tipos flotantes	`flotante16, flotante32, flotante64`
Otros tipos	`caracter, cadena, booleano`
Valores booleanos	`verdadero, falso`
Entrada y salida	`leer, mostrar`
Control de flujo	`si, sino, mientras, para`
Subprogramas	`fun, retornar, mainfun`

3. Decisiones al construir la gramática

Organizamos las producciones a partir del Laboratorio 1. En esta tabla resumimos las decisiones que tomamos.

Elemento	Decisión
Tipos	Usamos <Tipo_Dato> en lugar de <TIPO_BASICO>. Solo incluye tipos básicos, no clases.
Identificadores	Usamos `id` para los nombres. <Clase> y <Metodo> representan declaraciones completas.
Variables y vectores	Agrupamos su inicio común: `sea`, mutabilidad, nombre y `:`.
Declaraciones locales	Reutilizamos las declaraciones de variables y vectores, y añadimos objetos. Los objetos no se declaran globalmente.
Valores y operandos	<Literal> contiene literales sin signo numérico. <Valor> admite además números negativos. <Operando> también permite nombres, accesos, llamadas y expresiones entre paréntesis.
Asignaciones y llamadas	Agrupamos las instrucciones que empiezan con `id` o `este` y las distinguimos por lo que aparece después.
Modificaciones	Separamos las asignaciones completas de vectores y objetos de las modificaciones de atributos y posiciones de vector.
Retorno	<Tipo_Retorno> indica el tipo devuelto. <Retorno> representa la instrucción `retornar`.
Listas	Usamos λ cuando una lista puede estar vacía. Los atributos, métodos y valores de un vector requieren al menos un elemento.
Expresiones	Conservamos la prioridad de los operadores. Los operadores binarios de igual prioridad se interpretan de izquierda a derecha.

4. Gramática de FlowxyRust

$$G = (V_n, V_t, P, S)$$

$V_n = \{$

<S>, <Declaraciones_Globales>, <Constante>, <Declaracion_Variable_Vector>, <Contenido_Declaracion>, <Mutabilidad>, <Inicializacion>, <Inicializacion_Vector>, <Valor_Vector>, <Valores>, <Continuacion_Valores>, <Tipo_Dato>, <Entero>, <Decimal>, <Natural>, <Tipo_Entero>, <Literal>, <Numero>, <Valor>, <Clases>, <Clase>, <Atributos>, <Continuacion_Atributos>, <Declaracion_Atributo>, <Implementaciones>, <Implementacion>, <Metodos>, <Continuacion_Metodos>, <Metodo>, <Funciones>, <Funcion>, <Funcion_Principal>, <Parametros>, <Parametro>, <Continuacion_Parametros>, <Tipo_Retorno>, <Sentencias>, <Instruccion>, <Declaracion_Sea>, <Contenido_Declaracion_Local>, <Inicializacion_Objeto>, <Valor_Objeto>, <Valores_Atributos>, <Valor_Atributo>, <Continuacion_Valores_Atributos>, <Instruccion_Id>, <Continuacion_Instruccion_Id>, <Instruccion_Este>, <Continuacion_Instruccion_Miembro>, <Modificacion>, <Modificacion_Basica>, <Operador_Asignacion_Compuesta>, <Contenido_Asignacion>, <Asignacion_Tras_Id>, <Unaria_Sin_Id_Inicial>, <Continuacion_Expresion>, <Entrada>, <Salida>, <Destino>, <Continuacion_Destino>, <Retorno>, <Valor_Retorno_Opcional>, <Condicional>, <Sino_Opcional>, <Condicion>, <Mientras>, <Para>, <Inicializacion_Para>, <Actualizacion_Para>, <Expresion>, <Disyuncion_Logica>, <Continuacion_Disyuncion>, <Conjuncion_Logica>, <Continuacion_Conjuncion>, <Comparacion_Igualdad>, <Continuacion_Igualdad>, <Comparacion_Orden>, <Continuacion_Comparacion>, <Suma_Resta>, <Continuacion_Suma_Resta>, <Multiplicacion_Division_Modulo>, <Continuacion_Producto>, <Expresion_Unaria>, <Operando>, <Continuacion_Identificador>, <Llamada_Metodo_Opcional>, <Argumentos>, <Continuacion_Argumentos>

$\}$

Símbolos terminales: V_t

```
Vt = {
    sea, constante, mutable, clase, implementa, este,
    entero16, entero32, entero64,
    natural16, natural32, natural64,
    flotante16, flotante32, flotante64,
    caracter, cadena, booleano, verdadero, falso,
    leer, mostrar, si, sino, mientras, para,
    fun, retornar, mainfun,

    "-", "!", "+", "*", "/", "%",
    "<", "<=", ">", ">=", "==", "!=",
    "&&", "||", "=", "+=", "-=", "*=", "/=", "%=",
    "++", "--",

    "(", ")", "{", "}", "[", "]", ":", ";", ".", ",",

    id, literal_entero, literal_decimal,
    literal_cadena, literal_caracter
}
```

Símbolo inicial: S

S representa el programa completo: declaraciones globales, clases, implementaciones y funciones opcionales, seguidas de una única `mainfun` obligatoria.

Producciones: P

Ordenamos las reglas desde el programa completo hasta sus componentes. Las agrupamos por tema para facilitar su revisión, pero todas pertenecen al mismo conjunto P. Una regla puede usar otra que definimos más adelante.

$P = \{$

Programa y declaraciones globales

```
S → <Declaraciones_Globales> <Clases> <Implementaciones> <Funciones> <Funcion_Principal>

Declaraciones_Globales → <Constante> <Declaraciones_Globales>
    | <Declaracion_Variable_Vector> <Declaraciones_Globales>
    | λ

Constante → constante id : <Tipo_Dato> = <Valor> ;

Declaracion_Variable_Vector → sea <Mutabilidad> id : <Contenido_Declaracion> ;

Contenido_Declaracion → <Tipo_Dato> <Inicializacion>
    | [ <Tipo_Dato> ; literal_entero ] <Inicializacion_Vector>

Mutabilidad → mutable | λ

Inicializacion → = <Expresion> | λ

Inicializacion_Vector → = <Valor_Vector> | λ

Valor_Vector → [ <Valores> ]

Valores → <Valor> <Continuacion_Valores>

Continuacion_Valores → , <Valor> <Continuacion_Valores> | λ
```

Tipos y valores

```
Tipo_Dato → <Entero> | <Decimal> | <Natural> | cadena | caracter | booleano

Entero → entero16 | entero32 | entero64

Decimal → flotante16 | flotante32 | flotante64

Natural → natural16 | natural32 | natural64

Tipo_Entero → <Entero> | <Natural>

Literal → <Numero> | literal_cadena | literal_caracter | verdadero | falso

Numero → literal_entero | literal_decimal
```

```
Valor → <Literal> | - <Numero>
```

Clases e implementaciones

```
Clases → <Clase> <Clases> | λ

Clase → clase id { <Atributos> }

Atributos → <Declaracion_Atributo> <Continuacion_Atributos>

Continuacion_Atributos → <Declaracion_Atributo> <Continuacion_Atributos> | λ

Declaracion_Atributo → sea id : <Tipo_Dato> ;

Implementaciones → <Implementacion> <Implementaciones> | λ

Implementacion → implementa id { <Metodos> }

Metodos → <Metodo> <Continuacion_Metodos>

Continuacion_Metodos → <Metodo> <Continuacion_Metodos> | λ

Metodo → fun <Mutabilidad> id ( <Parametros> ) <Tipo_Retorno> { <Sentencias> }
```

Funciones y parámetros

```
Funciones → <Funcion> <Funciones> | λ

Funcion → fun id ( <Parametros> ) <Tipo_Retorno> { <Sentencias> }

Funcion_Principal → mainfun ( ) { <Sentencias> }

Parametros → <Parametro> <Continuacion_Parametros> | λ

Parametro → id : <Tipo_Dato>

Continuacion_Parametros → , <Parametro> <Continuacion_Parametros> | λ

Tipo_Retorno → : <Tipo_Dato> | λ
```

Instrucciones y declaraciones locales

```
Sentencias → <Instruccion> <Sentencias> | λ

Instruccion → <Constante>
    | <Declaracion_Sea>
    | <Instruccion_Id>
    | <Instruccion_Este>
    | <Entrada>
    | <Salida>
    | <Retorno>
    | <Condicional>
    | <Mientras>
    | <Para>

Declaracion_Sea → sea <Mutabilidad> id : <Contenido_Declaracion_Local> ;
```

```
Contenido_Declaracion_Local → <Contenido_Declaracion> | id <Inicializacion_Objeto>

Inicializacion_Objeto → = <Valor_Objeto> | λ

Valor_Objeto → id { <Valores_Atributos> }

Valores_Atributos → <Valor_Atributo> <Continuacion_Valores_Atributos>

Valor_Atributo → id : <Expresion>

Continuacion_Valores_Atributos → , <Valor_Atributo> <Continuacion_Valores_Atributos> | λ
```

Asignaciones y llamadas como instrucciones

```
Instruccion_Id → id <Continuacion_Instruccion_Id>

Continuacion_Instruccion_Id → <Modificacion> ;
    | [ <Expresion> ] <Modificacion_Basica> ;
    | ( <Argumentos> ) ;
    | . id <Continuacion_Instruccion_Miembro>

Instruccion_Este → este . id <Continuacion_Instruccion_Miembro>

Continuacion_Instruccion_Miembro → <Modificacion_Basica> ; | ( <Argumentos> ) ;

Modificacion → = <Contenido_Asignacion> | <Operador_Asignacion_Compuesta> <Expresion> |
    ++ | --

Modificacion_Basica → ++ | -- | = <Expresion> | <Operador_Asignacion_Compuesta>
    <Expresion>

Operador_Asignacion_Compuesta → += | -= | *= | /= | %=

Contenido_Asignacion → <Valor_Vector>
    | id <Asignacion_Tras_Id>
    | <Unaria_Sin_Id_Inicial> <Continuacion_Expresion>

Asignacion_Tras_Id → { <Valores_Atributos> }
    | <Continuacion_Identificador> <Continuacion_Expresion>

Unaria_Sin_Id_Inicial → <Literal>
    | este . id <Llamada_Metodo_Opcional>
    | ( <Expresion> )
    | - <Expresion_Unaria>
    | ! <Expresion_Unaria>

Continuacion_Expresion → <Continuacion_Producto> <Continuacion_Suma_Resta>
    <Continuacion_Comparacion> <Continuacion_Igualdad> <Continuacion_Conjuncion>
    <Continuacion_Disyuncion>
```

Entrada, salida y retorno

```
Entrada → leer ( <Destino> ) ;

Salida → mostrar ( <Expresion> ) ;

Destino → id <Continuacion_Destino> | este . id
```

```
Continuacion_Destino → [ <Expresion> ] | . id | λ

Retorno → retornar <Valor_Retorno_Opcional> ;

Valor_Retorno_Opcional → <Expresion> | λ
```

Condicionales y ciclos

```
Condicional → si ( <Condicion> ) { <Sentencias> } <Sino_Opcional>

Sino_Opcional → sino { <Sentencias> } | λ

Condicion → <Expresion>

Mientras → mientras ( <Condicion> ) { <Sentencias> }

Para → para ( <Inicializacion_Para> ; <Condicion> ; <Actualizacion_Para> ) {
    <Sentencias> }

Inicializacion_Para → sea mutable id : <Tipo_Entero> = <Expresion> | id = <Expresion>

Actualizacion_Para → id <Modificacion_Basica>
```

Expresiones y argumentos

```
Expresion → <Disyuncion_Logica>

Disyuncion_Logica → <Conjuncion_Logica> <Continuacion_Disyuncion>

Continuacion_Disyuncion → || <Conjuncion_Logica> <Continuacion_Disyuncion> | λ

Conjuncion_Logica → <Comparacion_Igualdad> <Continuacion_Conjuncion>

Continuacion_Conjuncion → && <Comparacion_Igualdad> <Continuacion_Conjuncion> | λ

Comparacion_Igualdad → <Comparacion_Orden> <Continuacion_Igualdad>

Continuacion_Igualdad → == <Comparacion_Orden> <Continuacion_Igualdad>
    | != <Comparacion_Orden> <Continuacion_Igualdad>
    | λ

Comparacion_Orden → <Suma_Resta> <Continuacion_Comparacion>

Continuacion_Comparacion → < <Suma_Resta> <Continuacion_Comparacion>
    | <= <Suma_Resta> <Continuacion_Comparacion>
    | > <Suma_Resta> <Continuacion_Comparacion>
    | >= <Suma_Resta> <Continuacion_Comparacion>
    | λ

Suma_Resta → <Multiplicacion_Division_Modulo> <Continuacion_Suma_Resta>

Continuacion_Suma_Resta → + <Multiplicacion_Division_Modulo> <Continuacion_Suma_Resta>
    | - <Multiplicacion_Division_Modulo> <Continuacion_Suma_Resta>
    | λ

Multiplicacion_Division_Modulo → <Expresion_Unaria> <Continuacion_Producto>
```

```
Continuacion_Producto → * <Expresion_Unaria> <Continuacion_Producto>
    | / <Expresion_Unaria> <Continuacion_Producto>
    | % <Expresion_Unaria> <Continuacion_Producto>
    | λ

Expresion_Unaria → - <Expresion_Unaria> | ! <Expresion_Unaria> | <Operando>

Operando → <Literal>
    | id <Continuacion_Identificador>
    | este . id <Llamada_Metodo_Opcional>
    | ( <Expresion> )

Continuacion_Identificador → [ <Expresion> ]
    | ( <Argumentos> )
    | . id <Llamada_Metodo_Opcional>
    | λ

Llamada_Metodo_Opcional → ( <Argumentos> ) | λ

Argumentos → <Expresion> <Continuacion_Argumentos> | λ

Continuacion_Argumentos → , <Expresion> <Continuacion_Argumentos> | λ

}
```

5. Programa de ejemplo

Reunimos las principales estructuras del lenguaje en el siguiente programa. Para revisarlo paso a paso, podemos empezar por `mainfun() { }` y agregar una declaración o instrucción cada vez.

```
constante MAYORIA : entero32 = 18;
sea mutable total : entero32 = 0;
sea mutable notas : [entero32; 3] = [10, 15, 20];

clase Persona {
    sea nombre : cadena;
    sea edad : entero32;
}

implementa Persona {
    fun obtenerEdad() : entero32 {
        retornar este.edad;
    }

    fun mutable cumplirAnios() {
        este.edad++;
    }
}

fun duplicar(numero : entero32) : entero32 {
    retornar numero * 2;
}

fun saludar(nombre : cadena) {
    mostrar("Hola " + nombre);
    retornar;
}

mainfun() {
    sea mutable persona : Persona;
    persona = Persona { nombre: "Diego", edad: 20 };

    sea mutable edadIngresada : entero32;
    leer(edadIngresada);
    persona.edad = edadIngresada;
    persona.cumplirAnios();

    sea edadActual : entero32 = persona.obtenerEdad();
    sea activo : booleano = verdadero;
    sea inicial : caracter = 'D';
    sea temperatura : flotante32 = -5.5;
    sea cantidad : natural32 = 3;
    sea resultado : entero32 = duplicar(edadActual);

    notas = [12, 16, 18];
    notas[0] = (notas[1] + notas[2]) / 2;

    si (edadActual >= MAYORIA && activo) {
        saludar(persona.nombre);
```

```
    } sino {
        mostrar("Registro pendiente");
    }

    sea mutable contador : entero32 = 0;
    mientras (contador < 3) {
        mostrar(notas[contador]);
        contador++;
    }

    para (sea mutable i : entero32 = 0; i < 3; i++) {
        total += notas[i];
    }

    // Mostrar el resultado acumulado.
    mostrar(total);
    mostrar(resultado);
}
```